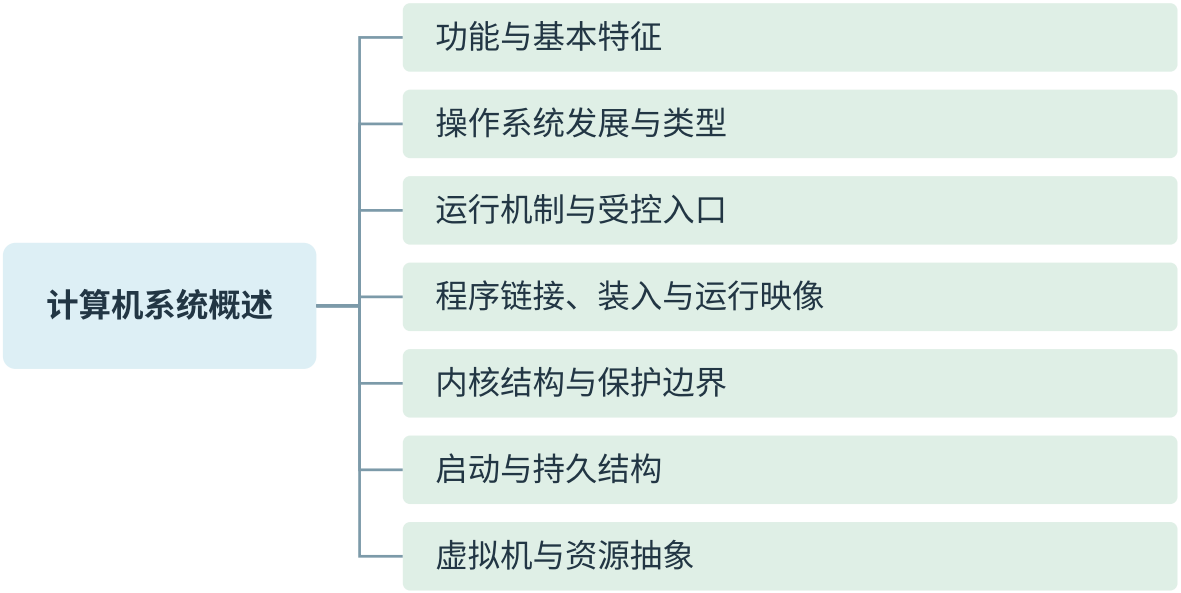
计算机系统概述 1

章节脉络



课程连接·运行环境

硬件资源由内核管理；用户程序通过受控入口申请服务，权限级与地址空间提供不同保护。

1.1 资源、权限与控制流

概念辨析的四步路径

1. 先确定对象：是硬件资源、内核结构、用户程序，还是磁盘上的持久数据？
2. 再确定权限：操作能否影响其他进程或全局保护状态，是否需要受控入口？
3. 沿一次执行追踪：普通调用、陷入、异常响应、软件处理、返回，各由谁完成？
4. 最后检查条件：并发不自动推出并行，模式切换不自动推出进程切换，结构收益也要说明代价。

定义 1.1.1 (用资源与接口理解操作系统)

操作系统对处理器、内存、文件和设备实行受保护的管理，并向应用提供服务。学习时同时标出“管理对象”“允许谁操作”“通过哪个入口请求”；基本特征和内核结构都围绕这三件事展开。

1.1.1 受控入口为何不能由用户代码自行授予权限

从保护目标推出入口约束

假设用户程序能先用普通赋值把模式位改为内核态，再调用服务，那么它也能跳过服务的参数检查，直接关闭时钟中断或改写别的进程映射，资源隔离立即失效。因此权限变化必须由硬件按预定事件与入口

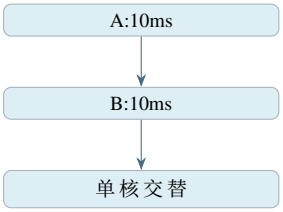
规则执行；内核取得控制权以后再检查请求。用户可以发起请求，并不等于用户能决定入口与授权结果。

□

程序执行一条普通加法指令时，通常直接使用处理器；读取文件时，却不能自己任意驱动磁盘。两种操作的差别来自资源保护：处理器、内存和设备由多个程序共享，操作系统必须管理它们，并给应用提供受控的使用接口。本章沿着一个程序的运行过程，梳理用户态、内核态、异常、系统调用和地址空间之间的关系。

最小反例·单核也有并发

一个 CPU 先运行 A 十毫秒，再运行 B 十毫秒，二者在同一时间段交替推进，已满足并发，却没有在同一时刻执行两条用户指令。把“系统有多个任务”直接推出“任务必并行”，就混淆了时间区间与时间点。



同一时间段交替推进是并发，不推出同一时刻指令并行。

1.2 操作系统管理什么，向程序提供什么

操作系统是控制和管理计算机硬件、软件资源，组织程序运行，并向用户和应用程序提供接口的系统软件。这里的“管理”要落到具体对象上：处理机管理决定谁运行；内存管理分配空间并隔离访问；文件管理组织持久数据；设备管理协调输入输出，屏蔽设备差异。

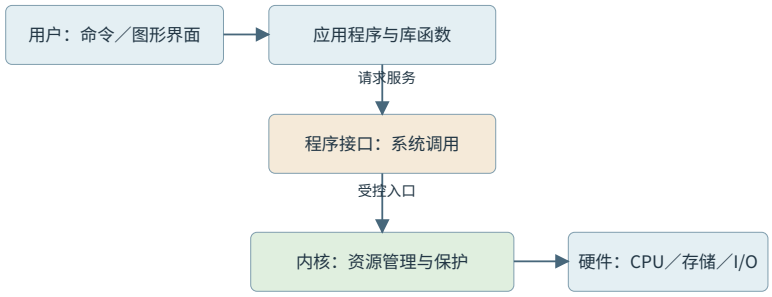
面向用户的命令行、图形界面便于发起操作；面向应用程序的系统调用接口允许程序请求内核服务。图形界面不是所有操作系统都必须具有的组成部分。编译器虽然也是系统软件，但把高级语言翻译成目标程序并不是操作系统的基本职责。

〖1.1-003〗下列哪项不是操作系统关心的基本问题：管理裸机、提供用户程序与硬件的接口、管理系统资源、提供高级语言编译器？答案是最后一项。判断依据是该功能是否承担资源管理与运行控制，不能只看它是否属于“系统软件”。

1.2.1 用户接口：联机命令、脱机命令与程序接口

操作系统向上提供的接口要区分使用者和使用方式。**联机命令接口**又称交互式命令接口，用户输入一条命令，系统解释并执行，再接收下一条；**脱机命令接口**又称批处理命令接口，用户预先组织一组作业控制命令，系统按约定顺序处理，执行期间通常不需要用户逐条干预。两者的“联机／脱机”描述的是命令提交与交互方式，不直接等同于有没有网络连接。

程序接口由一组系统调用构成，供程序请求服务。用户在文件管理器中点击“删除”属于使用图形用户界面，应用程序随后可能通过系统调用完成文件操作。图形界面、命令解释器、库函数和系统调用分别处在不同层次；把图形界面的一次点击直接视为 CPU 执行一条机器指令，会漏掉应用处理、库封装和内核服务。



命令接口、库函数、系统调用与机器指令属于不同层次。

Figure 1.1: 用户通过命令或图形界面使用应用，应用通过程序接口请求内核服务

概念辨析·脱机并不等于离线网络

联机与脱机命令接口按交互提交方式区分。批处理命令文件可以由联网计算机执行，分类依据仍是预先组织命令后批量运行。

操作系统对硬件进行组织与抽象，把原本不便直接使用的裸机扩充为便于使用的机器。因此“扩充机器”强调增加服务与抽象，并不表示凭空增加 CPU、内存或磁盘的物理容量。硬件资源仍有限，资源管理目标还包括安全、有效和公平地组织使用。

例如应用中的 `printf` 先进行格式转换，输出路径可能再调用系统服务；计算绝对值的库函数可以只执行用户态指令。判断接口类型时，应沿实际执行路径辨认，而不是依据函数名长短或是否属于标准库。

1.2.2 并发、共享、虚拟与异步怎样联系

并发指多个活动在同一时间间隔内推进；**并行**指多个活动在同一时刻执行。一个 CPU 交替执行两个进程，已有并发；多个核上分别执行，才有处理器层面的并行。单 CPU 的计算与设备的数据传输仍可能并行。

共享指资源供多个并发活动共同使用。打印机一类资源通常需要互斥使用；磁盘文件或内存中只读代码可按规定共享。所谓“同时共享”不等于所有硬件操作都在同一瞬间发生，而是允许多个活动在同一时期使用同一资源。

虚拟把有限的物理资源抽象为若干逻辑资源。例如时间片让每个用户似乎独占一个处理器，虚拟内存让进程使用独立的地址空间。**异步**指进程因竞争处理器、等待 I/O 等原因，以不可预先确定的速度向前推进。它不表示进程内部可以违背程序依赖，也不等同于接口中的“异步 I/O”。

并发活动会竞争资源，因而共享必须伴随保护；虚拟化为并发活动提供合适的使用视图；资源等待又造成执行速度的不确定。后续同步与调度问题都以这些关系为前提。

1.2.3 例 1-1　进程越多，处理器利用率一定越高吗

例 1.2.1 (配套例题)

【配套例题】下列关于多道程序系统的叙述中，不正确的是（　）。

A. 支持进程的并发执行　　B. 不必支持虚拟存储管理

C. 需要实现对共享资源的管理　　D. 进程数越多 CPU 利用率越高

解析。多道程序设计的直接收益是：一个进程等待 I/O 时，CPU 可

以运行其他就绪进程。但这一收益有前提：可运行进程过多会增加调度开销和内存压力，严重时频繁换页，CPU 反而难以有效执行用户计算。因此 D 把条件性收益写成了单调关系，答案为 **D**。多道程序可以使用不带请求分页的内存方案，B 正确；“多道” 不等于 “必须有虚拟内存”。

错项排除·实时性的判据是期限

控制任务必须在五毫秒内响应，即使某机器多数时候只用一毫秒，偶尔延迟十毫秒也可能违反硬实时要求。排除 “实时系统就是运行速度最快的系统” 时，应指出缺少最坏响应与截止期限这一保证。

1.3 批处理、分时与实时解决不同的问题

早期人工装入程序时，CPU 常等待操作者准备下一项作业。单道批处理把多个作业预先排好，自动逐个执行，减少人工干预；但一个作业等待 I/O 时，处理器仍可能空闲。多道批处理允许多项作业同时在内存中，利用 CPU 与 I/O 的重叠提高吞吐量。中断使设备完成时能够通知 CPU，存储保护、调度和资源管理使多道运行可控。

分时系统把处理器时间分成时间片，轮流服务交互用户，主要关注响应时间和交互性。一个时间片并不等于某个程序的全部运行时间；未运行完的程序会在以后继续执行。**实时系统**关注任务能否在规定期限内得到响应或完成。硬实时任务错过截止时间便不能满足系统要求；软实时允许一定程度的超期，但质量会下降。实时性不能只解释成 “运行得快”：提前 1 秒完成与晚于期限 1 毫秒完成，两者的结果差异可能与绝对执行速度无关。

类型	主要组织方式	评估时首先关心什么
多道批处理	作业成批提交，运行中较少交互	吞吐量、周转时间、资源利用率
分时	多用户交互，时间片轮转	响应时间、使用公平性
实时	按期限和任务紧迫性组织执行	截止时间、响应的可预测性

网络操作系统支持联网计算机上的资源与通信服务；分布式系统进一步组织多台计算机协作，可能提供统一资源视图。单用户、多用户、单任务、多任务则是另外的分类维度，不能把 “单用户” 自动理解为 “单任务”。

1.3.1 从人工操作到多道程序：CPU 为什么会空闲

人工操作阶段，装入纸带、检查结果和准备下一作业需要人参与，CPU 常等待外部准备。脱机输入／输出把部分慢速操作交给外围设备预先处理，减少主机等待人工操作的时间；单道批处理进一步由监督程序自动安排作业，但内存中仍只有一道作业可运行，该作业等待 I/O 时，CPU 缺少其他可切换的工作。

多道程序设计把多道作业同时放在内存中：某道作业等待设备时，另一道就绪作业可以使用 CPU。这里有两种不同的重叠：CPU 与 I/O 设备可以物理并行；同一 CPU 上的多道程序交替执行形成并发。收益来自填补空闲时间，而系统同时承担处理机分配、存储保护、设备协调等额外管理工作。

若只有进程 A，A 依次计算 2 ms、输入 4 ms、计算 2 ms，忽略系统开销，则完成时间 8 ms，CPU 有效计算 4 ms，利用率为 50%。若在 A 等待输入时运行另一个就绪进程 B 的 4 ms 计算段，时间线可成为 A 计算 0～2、A 输入与 B 计算 2～6、A 计算 6～8；总历时仍为 8 ms，却完成 8 ms 计算，CPU 利用率达到 100%。这是根据上述机制构造的示例，成立前提是设备与 CPU 可并行、B 始终就绪且忽略切换开销。

批处理主要追求吞吐量与周转；分时进一步强调交互、独立使用感和及时响应。实时系统还必须依据任务期限安排执行，硬实时与软实时

在能否容忍超期方面不同。几类系统可在现代实现中兼具若干特征，分类是在比较主要设计目标。

最小反例·内核也要做普通加法

内核计算页框号或更新计数器时，会执行普通算术指令。它们在用户态通常也合法，所以 “内核态执行的都是特权指令” 不成立。限制方向是特权操作要求高权限，不能把这个蕴含关系反过来。

1.4 用户态、内核态与特权指令

处理器的**工作模式**规定当前代码能够执行哪些受保护操作。用二态模型表示时，用户态权限较低，内核态权限较高。模式位保存在程序状态信息中，由硬件按规则改变；普通用户程序不能靠一条赋值指令就把自己切换成内核态。

特权指令只能在规定的高权限模式执行，例如修改关键内存映射、设置中断屏蔽状态、直接控制受保护 I/O。若允许任意用户程序关闭时钟中断，它就可能长期占用 CPU，使调度失去控制。普通算术、逻辑和多数寄存器操作没有这种系统级影响，可以在用户态执行，也可以在内核态执行。

由此要区分两件事：特权指令必须在内核态运行，但内核态运行的并不都是特权指令。前者约束的是特权指令；若把内核态理解成只能执行特权指令，内核中的普通计算就无从进行。陷入指令是用户程序主动进入受控入口的指令；它允许在用户态执行，执行后由硬件转入规定的内核入口，不能据 “会进入内核” 反推它是用户态禁止执行的特权指令。

1.5 中断和异常怎样把控制权交给内核

外部中断由正在执行的指令以外的事件引起，如时钟到期、设备完成。**异常**由当前指令执行过程中的事件引起，如除数为零、缺页、执行非法指令；主动执行陷入指令也通过受控异常入口请求服务。不同教材可能把 “中断” 作为包括内外事件的总称，本书需要区分时使用 “外部中断” 和 “异常”。

一次典型控制转移包括以下连续过程：硬件检测事件，保存继续执行所需的断点和程序状态，按事件号找到处理入口并切换到所需模式。入口代码再保存会被处理程序改写的通用寄存器，然后执行处理；退出时恢复相应现场，使用专门返回指令恢复程序状态和继续位置。具体保存位置以及硬件、软件的分工取决于体系结构，不能把所有现场的保存都归给同一方。

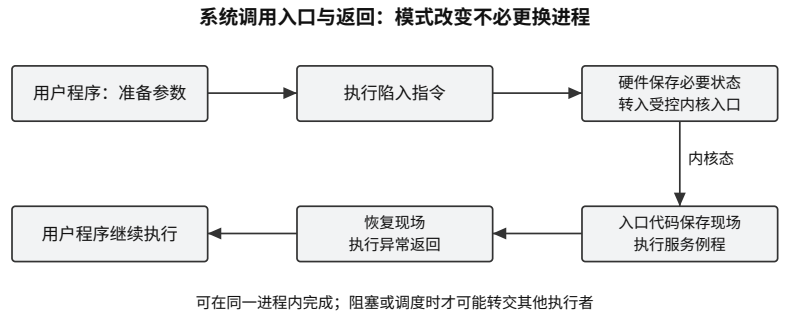


Figure 1.2: 从用户程序进入内核，返回时仍可继续原进程

外部中断通常在指令边界被响应；异常通常在指令执行期间被发现。可恢复故障处理完后，可能重新执行产生异常的指令，例如缺页后重试访存；陷入返回通常从陷入指令的后继指令继续执行；不可恢复错误可能终止进程。题目给出具体指令集时，以其断点与返回规则为准。

1.5.1 陷阱、故障与终止：异常返回地址为何不同

错项排除·系统调用不保证阻塞

读取的数据若已经在内核缓存且可立即返回，系统调用仍跨越了保护边界，却可能始终由原进程完成。判断阻塞要看所需事件是否尚未满足，判断进入内核要看是否通过受控入口，两项不能互相替代。

异常可按触发与恢复方式区分。**陷阱（trap）**通常是程序有意发起的受控事件，常用于请求内核服务，处理完后继续执行后继指令。**故障（fault）**表示当前指令执行条件暂不满足；若处理能够修复条件，如为合法虚拟页调入内容，返回后可重新执行引起故障的指令。**终止（abort）**表示难以恢复的严重事件，通常无法正常继续原来的指令序列。

类别	事件与当前指令的关系	常见后续行为
外部中断	来自设备、时钟等外部事件	处理后恢复被打断程序，或依据调度决定切换
陷阱	程序有意执行陷入指令	从规定的后继位置继续
故障	当前指令执行发现可处理问题	修复条件后重试原指令
终止	发生难以恢复的严重错误	结束相关执行或进入故障处理

分类用来说明机制，不宜只凭“除零”等事件名称断言所有体系结构的处理和返回方式完全相同。具体题目给出指令集、异常模型或操作系统行为时，应以题设为准。

中断向量提供中断处理入口等信息；**中断向量表**按事件编号组织这些入口信息。响应机制定位入口后，才进入相应的软件处理程序。中断响应、保存现场、执行处理、恢复现场和返回是连续环节，硬件与软件的职责应分别核对。

1.5.2 例 1-2　执行系统调用时，哪些操作属于软件

例 1.5.1 (配套例题)

【配套例题】执行系统调用的过程涉及下列操作，其中由操作系统完成的是（　）。

I. 保存断点和程序状态字　　II. 保存通用寄存器的内容

III. 执行系统调用服务例程　　IV. 将 CPU 模式改为内核态

A. 仅 I、III　　B. 仅 II、III

C. 仅 II、IV　　D. 仅 II、III、IV

解析。按题目采用的常规异常入口模型，I、IV 由响应机制中的硬件完成；II 由内核入口代码完成，III 本来就是内核软件执行的服务。因此答案为 **B**。将模式切换留给不受保护的用户代码会破坏入口的受控性；内核不能等到自己已经进入服务代码，才由普通指令任意决定是否取得权限。

中断处理不必导致进程切换。例如时钟中断只更新计数后返回原进程，模式虽然来回切换，但运行的仍是原进程。这个区别在第 2 章用于分析上下文切换。

1.6 系统调用与库函数：调用一段代码，还是请求内核服务

系统调用是应用通过规定接口请求内核服务的过程。程序准备调用号和参数，执行陷入指令；内核检查参数、权限及资源状态，分派到相应

服务；服务完成时，或进程阻塞后又被唤醒继续时，内核把结果返回用户程序。检查参数不能省略，例如用户提供的缓冲区地址必须落在允许访问的范围内。

库函数是程序可以调用的已有函数。纯字符串长度计算可以完全在用户态完成；文件读写库函数可能整理格式、维护用户缓冲区，再调用 `read`、`write` 等系统调用。因而“调用库函数”既不必然发生模式切换，也不排斥发生系统调用。反过来，系统调用也不是普通函数调用：它跨越保护边界，并通过内核规定的入口执行。

读取磁盘文件时，`read(fd, buf, n)` 中的 `fd` 标识已经打开的文件，`buf` 是接收数据的用户缓冲区，`n` 是请求字节数。若数据已在内核缓存，内核可能复制后直接返回；若必须等待设备，当前进程可能先阻塞，CPU 转去运行其他进程。不要把“系统调用进入内核”与“系统调用必阻塞”合为一条结论。

〖1.1-010〗库函数不直接处理所有受保护硬件请求。判断一个具体函数是否引起陷入，要看它在本次路径上是否请求内核服务，不能只凭“库函数”这个名称。

不变量·先判范围，再求和

界限寄存器按长度定义时，合法逻辑地址始终满足 $0 \le a < L$。即使 $B + a$ 仍落在已安装的物理内存内，也不能放行 $a = L$。这个不变量保护的是当前进程的合法区间，与整台机器是否还有空间无关。

1.7 从源程序到可运行映像：链接、装入与地址绑定

编译把源程序翻译为目标模块，目标模块可能仍含未确定的外部符号。**链接**把多个目标模块和所需库连接起来，解析符号并处理地址引用，形成可装入文件。静态链接在运行前完成所需模块的合并；装入时动态链接在装入时连接；运行时动态链接在首次调用等运行时机连接所需模块。

装入把可执行文件需要的内容映射或读入进程地址空间，并建立运行条件。地址绑定可能发生在不同阶段。

方式	地址何时确定	移动到另一个物理位置会怎样
绝对装入	编译或汇编时已给定绝对地址	通常须重新生成或修补地址
可重定位装入	装入时按实际起点修改相关地址	装入后不能仅换一个基址就随意搬移
动态运行时装入/重定位	执行访存时由地址变换机构转换	更新映射后可改变物理位置，需相应硬件支持

最小的动态重定位模型使用基址寄存器 B 和界限 L。程序产生逻辑地址 a，先检查 $0 \le a < L$，合法时物理地址为 $B + a$。例如 $B = 12000, L = 2000$，逻辑地址 1500 转为 13500；逻辑地址 2000 已越界，不能因为“两数相加仍在主存范围内”就判为合法。界限表示长度时，上界通常不包含；若题目把寄存器定义成最大合法地址，则应按其定义调整比较。

动态重定位在执行阶段进行，依赖重定位信息与硬件地址变换机构，不需要每次访存都调用装入程序。操作系统负责设置和维护映射，硬件完成运行中的快速转换。分页的非连续映射在第 3 章展开。

1.8 进程地址空间中，代码、数据、堆和栈各放什么

课程连接·指针自身与所指对象分开

数据结构中的链表头指针可能是函数局部变量，所指结点却由动态分配得到；函数退出与结点释放属于不同生存期问题。分析内存图时为“指针变量”和“动态对象”分别画位置，能避免把整个链表误判为栈对象。

地址空间是一组可由地址标识的位置。进程的虚拟地址空间提供其运行时看到的地址视图，不要求这些地址对应的内容全部同时存在物理内存。可执行文件的磁盘布局、进程的虚拟布局、物理页框布局是三个不同对象。

典型用户空间包含代码区、静态数据区、堆、栈和映射区。代码区存放机器指令，通常只读且可共享；已初始化的全局变量、静态变量通常在初始化数据区，未显式初始化或零初始化的数据可用零填充区表示。堆用于动态申请的对象，栈保存函数活动记录中的参数、返回信息与未被优化掉的自动局部变量。共享库和映射文件可位于映射区。具体地址增长方向和排列由平台决定，示意图不能代替题设。

考虑以下代码，假定采用常规 C 运行时布局且忽略编译优化：

```c
#include <stdlib.h>

int g = 3;
static int z;
int main(void) {
    int n = 16;
    char *p = malloc(n);
    /* 使用 p 指向的空间 */
    free(p);
}
```

`g` 是初始化静态存储期数据，`z` 是零初始化静态存储期数据；`n` 和指针变量 `p` 自身属于 `main` 的局部活动记录；`p` 指向的动态对象属于堆。指针存在哪里与它指向的对象存在哪里必须分别回答。“写在函数外”并不等于“属于代码段”；局部变量也未必都放在栈上，优化可能把它们放进寄存器或直接消除。

进程控制块 PCB 保存调度与管理信息，是内核数据结构，不是应用能够随意修改的用户栈对象。同一程序的两个运行实例可以共享只读代码，但具有不同的进程身份、逻辑状态和数据视图。

1.9 内核结构：把哪些功能放进同一保护域

分层结构按依赖关系划分功能，上层使用规定的下层服务，便于逐层验证；代价是层次难划分、调用路径可能增长。**模块化结构**把功能组织为接口清楚的模块，允许在约定接口下替换实现。模块划分不自动规定严格的层次调用关系，模块也不等于独立的保护域。模块设计追求模块内部功能紧密相关（高内聚），模块之间通过尽量少的依赖协作（低耦合）。接口需要在开发前充分设计；若需求尚不明确，预先规定所有接口较困难，跨模块协调也可能增加设计和维护成本。

宏内核把较多的管理功能放在内核地址空间中，功能间可以直接调用，通信开销较小；一个内核模块的错误也可能影响整个系统。**微内核**把最基本的调度、低级地址空间管理、进程间通信等保留在内核，把较多服务移到用户态服务器。隔离与替换更容易，但一次服务可能增加消息传递和保护域切换，不能无条件断言性能更好。

外核进一步把资源保护和复用机制与高层资源抽象分开，让应用

或库操作系统按需要实现抽象。它并非 “没有内核” 或 “不保护资源”；恰恰相反，保留必要的保护职责正是外核的基础。

比较宏内核与微内核时，先看服务放在哪个保护域，再分析通信和故障如何跨越这个边界。下表比较典型结构；实际系统可以混合采用这些设计。

共同维度	宏内核	微内核
服务的位置	文件系统、设备管理等较多服务在内核态	只保留基本机制，较多服务由用户态服务器实现
服务间通信	同一内核地址空间内可直接调用	常经进程间通信、消息传递和保护域切换
故障影响	内核模块错误可能破坏其他内核功能	用户态服务器的故障较易隔离，但仍需设计恢复机制
扩展和移植	模块接口有助维护，修改仍须考虑内核内部依赖	可分别调整服务器；小内核较便于验证、移植
性能代价	调用路径较短；功能多不等于任意实现都快	跨域通信可能增加开销；隔离收益不能直接推出性能更高

适用边界·关中断的作用范围

当前 CPU 暂不响应可屏蔽中断，不会自动阻止另一 CPU 访问共享结构。多处理机原子性还依赖锁、原子指令等机制。

1.9.1 例 1-3　微内核的收益与代价

例 1.9.1 (配套例题)

【配套例题】与宏内核操作系统相比，下列特征中，微内核操作系统具有的是（　）。

I. 较好的性能　II. 较高的可靠性　III. 较高的安全性　IV. 较强的可扩展性

A. 仅 II、IV　　B. 仅 I、II、III

C. 仅 I、III、IV　　D. 仅 II、III、IV

解析。服务间保护边界有利于限制故障影响，减少高权限代码有利于安全，服务模块也更容易扩充；但跨域通信会增加开销，不能把 I 列为一般优势。答案为 **D**。这些是结构层面的比较，不是对任意两个具体系统的性能或安全性作无条件排名。

1.9.2 原语、时钟与内核的基本机制

内核的底层机制通常包括时钟管理、中断处理和原语。时钟中断既可支持计时，也为时间片到期、定时等待等管理提供事件来源。原语负责完成需要保持一致性的关键操作，例如保存现场、改变进程状态和移动 PCB 队列。

原语的**原子性**要求其他执行者不能观察到破坏约定的中间状态。单处理器模型常用短暂关闭可屏蔽中断的方法保护关键操作；它的作用范围是当前处理器，且不能屏蔽所有可能的异常。多处理器上，别的处理器仍可能访问共享数据，因此还需要锁或原子指令等同步机制。不能把 “本 CPU 不响应某类中断” 当作 “整台机器没有并发访问”。

分层结构、模块化、宏内核和微内核比较的是功能组织和保护边界。模块之间接口清楚不意味着每个模块都必须独立运行在用户态；微内核将服务移出高权限域，也仍需通过内核机制保护通信和共享资源。

1.10 开机以后，内核怎样获得控制权

课程连接·开机结构与磁盘格式不同

中断入口、页表和调度队列属于本次运行所需结构；分区表、根目录和磁盘索引节点通常是持久存储的一部分。把题中对象按“随运行建立”与“跨重启保留”定位，才能判断初始化是否需要重新创建。

典型引导链为：处理器复位后执行固件中的启动代码，固件初始化必要硬件并选择启动设备；引导程序定位并装入内核；内核建立中断入口、内存管理和设备管理等基本环境，创建初始进程，最后进入正常调度与用户服务。引导程序可以分阶段，早期阶段只装入后续阶段，不能把所有启动代码都理解成同一段磁盘程序。

传统 BIOS/MBR 模型中，磁盘主引导记录可含引导代码和分区表；现代 UEFI/GPT 的组织不同。回答“引导程序在哪里”必须先确定所指的是固件初始代码、主引导代码，还是文件系统中的后续装载器。引导不等于格式化：已经安装的文件系统和分区信息可以跨多次开机保留，而内核运行所需的内存管理结构需要本次启动建立。

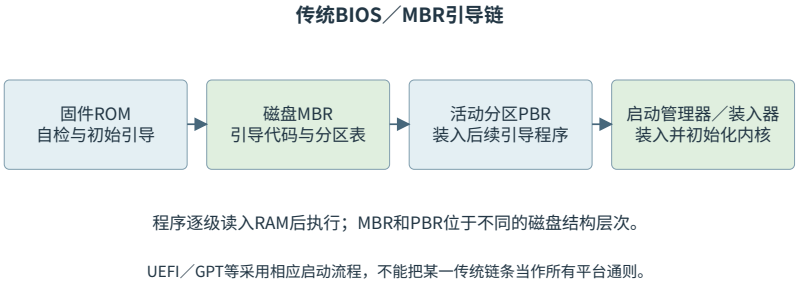


Figure 1.3: 传统 BIOS ／ MBR 模型中，固件、MBR、PBR 和后续装入程序逐级建立启动环境

1.10.1 例 1-4　每次开机需要重新建立什么

例 1.10.1 (配套例题)

【配套例题】下列选项中，需要在操作系统进行初始化过程中创建的是（　）。

A. 中断向量表　　B. 文件系统的根目录

C. 硬盘分区表　　D. 文件系统的索引节点表

解析。中断向量表是本次内核运行所需的控制转移结构，需要在初始化中建立，答案为 **A**。其余为磁盘格式与文件系统的持久结构，通常在分区或创建文件系统时形成，不应每次开机重建。

1.11 虚拟机：客户操作系统也可以成为被管理者

虚拟机管理程序 VMM 向客户操作系统提供虚拟处理器、内存和设备，安排多个虚拟机使用物理资源，并隔离其影响。客户操作系统中的“内核态”只是客户的权限视图，并不意味着客户操作系统可以越过 VMM 任意修改主机资源。敏感操作由硬件虚拟化支持、陷入模拟或其他受控机制处理，具体路径随实现变化。

第一类 VMM 直接运行在硬件上；第二类 VMM 依托宿主操作系统运行。二者都可能采用硬件加速，因此不能把它们简单等同于“硬件实现”和“软件实现”。多台虚拟机可以在单核上交替运行，也可以在多核上并行运行，所以“同时存在多台虚拟机”不能推出它们在每一时刻都真正并行。

进程虚拟地址空间隔离的是进程访存；系统虚拟机为客户操作系统提供完整机器抽象。容器通常共享宿主内核，通过名称空间、资源控制等隔离进程环境，但这不等同于每个容器都带一个独立客户内核。这里借容器辨析抽象层次，不要求记忆其实现细节。

抽象层次·客户与宿主分开判断

客户进程阻塞是客户操作系统内部事件；虚拟机暂停是整个客户机器失去物理运行机会。二者的管理者和影响范围不同。

1.11.1 虚拟机的两种组织方式

维度	第一类 VMM	第二类 VMM
直接依托的层次	直接运行在物理硬件上	作为宿主操作系统上的程序运行
设备服务路径	由 VMM 及其设备服务组件组织	还利用宿主系统的设备驱动与服务
客户系统看到的对象	虚拟 CPU、虚拟内存、虚拟设备	同样是完整的虚拟机器抽象
存储资源分配	由 VMM 管理物理存储并分配客户空间	虚拟磁盘常表现为宿主文件系统中的文件
性能与可支持数量	少一层宿主转接，通常具有较小的额外开销	还需宿主资源；数量与性能受具体负载和实现影响
虚拟机迁移	依赖管理程序的镜像、设备和迁移机制	虚拟磁盘文件便于复制；仍须匹配虚拟硬件与平台
运行权限	VMM 管理物理资源，特权机制由平台提供	可由用户态组件与宿主内核组件协同完成

客户内核使用虚拟 CPU 的高权限操作时，实际保护层仍须确认其影响只落在分配给该虚拟机的资源上。虚拟机暂停与客户进程阻塞不是同一个层次的状态：前者可使整个客户系统暂时没有物理 CPU 运行机会，后者是客户系统内部某个执行者在等待事件。



Figure 1.4: 第一类 VMM 直接依托硬件；第二类 VMM 使用宿主操作系统提供的服务

第一类 VMM 直接管理物理资源，可把物理存储空间等划分给客户系统；第二类 VMM 借助宿主文件、内存和设备服务，客户虚拟磁盘可以表现为宿主文件中的数据。通常第一类少一层宿主转接、资源控制更直接；第二类依赖宿主的资源与驱动，作为普通宿主文件保存的虚拟机器也较便于复制迁移。实际性能、客户种类和迁移能力还取决于硬件虚拟化支持及产品实现。

虚拟化还必须控制客户内核的特权操作：客户看到的高权限不等于可以越过 VMM 访问别的客户资源。支持虚拟化的 CPU 提供相应的执行与陷入机制；传统 Ring 0～3 权限示意中的 Ring 0 与用户 Ring 3，

说明的是受控权限等级，不能据此断定客户内核直接取得整台物理机的最高控制权。

课程连接·回忆顺序

先区分用户态与内核态，再分别说明异常入口、内核操作和返回的位置。

1.12 分支回顾与自测

以下为原创回顾设问，用于检查本章机制与边界。

1. 普通函数调用、系统调用、设备中断是否都必然更换当前进程?
2. 用户程序执行陷入指令后进入内核，是否说明陷入指令应禁止在用户态执行?
3. 基址为 4096、界限为 1024，逻辑地址 1023 与 1024 分别怎样处理?
4. 指针变量在栈上，是否说明它指向的对象也在栈上?
5. 单核运行四台虚拟机，是否要求四台同时执行机器指令?

1.12.1 自测答案

1. 都不必然。函数调用通常不跨保护域；系统调用和设备中断可能只切换模式后返回原进程。只有实际选中另一个进程时，才发生调度切换。
2. 否。陷入是用户可执行的受控入口；目标入口和权限变化受硬件及内核规则约束。
3. 1023 合法，物理地址为 5119；1024 等于长度上界，越界。
4. 否。须分别判断指针自身与所指对象的生存期和分配方式。
5. 不要求。它们可以分时复用一个处理器，并发而非同一时刻并行。

1.13 主动回忆自测

以下均为自拟题，先盖住答案，按 “对象—权限—执行路径—条件” 口述理由。

功能、权限与结构的综合辨析

1. 单 CPU 交替运行两个程序，哪些特征成立？能否据此断言两程序指令并行?
2. 为什么增加进程数不一定持续提高 CPU 利用率?
3. 陷入指令为何能在用户态执行，而修改中断屏蔽状态通常不能?
4. 一次设备中断后仍返回原进程，发生了什么切换，又没有发生什么切换?
5. 基址 12000、界限 2000，逻辑地址 1999 与 2000 怎样处理?
6. 函数局部指针指向 malloc 得到的空间，两个对象各属于哪种存储?
7. 微内核减少高权限代码，为什么仍不能直接断言性能更高?
8. 第一类、第二类 VMM 的区别是什么？单核运行多台虚拟机是否矛盾?

1.13.1 核对答案与关键理由

1. 同一时间段交替推进是并发；单核同一时刻通常只执行一条用户指令，不能据此判用户指令并行。
2. 可隐藏部分 I/O 等待，但过多进程会增加调度、同步和内存压力，严重时抖动，收益有条件。

3. 陷入只进入预定的受控入口；任意用户修改全局屏蔽状态会破坏调度与保护，因而受权限约束。
4. 中断可导致用户态与内核态之间的模式变化；实际运行身份仍是原进程时，没有切换到另一个进程。
5. 1999 满足 $0 \le a < 2000$，物理地址 13999；2000 等于长度上界，越界，不能先相加再放行。
6. 指针变量本身通常位于函数活动记录中，动态对象在堆；优化可能改变局部变量的实际存放，题设优先。
7. 服务移出内核改善隔离，却可能增加 IPC、消息复制和保护域切换；不同实现的总性能须实际比较。
8. 第一类直接运行于硬件，第二类依托宿主系统；二者都可用硬件加速。多虚拟机可分时复用单核，无矛盾。

课程连接·自主检验

遇到调用流程题，依次写出发起者、特权转换、内核处理和返回条件。